

ALLIANCE SCHOOL OF ADVANCED COMPUTING

Department of IT, ASAC

ASSIGNMENT 2

Prim's (MST) : Special Subtree

Course Name: Data Structures and Algorithms

Course Code: IE1ITC302

Assignment No: 3

Name of the Student: Kaaysha Rao

Registration Number: 2411021060775

Platform: HackerRank

Language: Java

Algorithm: Prim's Algorithm (Minimum Spanning Tree)

A. Title of The Problem:

Prim's (MST) : Special Subtree

B. Problem Statement in details:

Given a weighted, undirected graph consisting of several edges connecting its nodes, find a subgraph that contains all the nodes, has the minimum possible total edge weight, and has exactly one path between any two nodes. Such a subgraph is called a Minimum Spanning Tree (MST). The MST is constructed using Prim's Algorithm starting from a specified start node. The required output is the total weight of all selected MST edges.

At every step, Prim's Algorithm selects the minimum-weight edge that connects a visited vertex to an unvisited vertex. The process continues until all vertices are included in the spanning tree. The graph may contain multiple edges between the same pair of nodes, so each edge is considered independently.

C. HackerRank/Leetcode Link of the problem (If applicable):

[Prim's \(MST\) : Special Subtree | HackerRank](#)

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

[Next Challenge](#)

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message

Success

Input (stdin)

```
1 5 6
2 1 2 3
3 1 3 4
4 4 2 6
5 5 2 2
6 2 3 5
7 3 5 7
8 1
```

[Download](#)

D. Student's Github link of report and code upload, along with the readme file:

[Graph-Analytics/Assignment 3 at main · KaayshaRao20/Graph-Analytics](https://github.com/KaayshaRao20/Graph-Analytics/Assignment%203)

The Assignment2 folder contains the Java source code, README documentation, and the HackerRank sample-test/output evidence for the Prim's MST problem.

E. Solution Steps/Algorithm:

1. Read n , m , the m weighted edges, and the starting vertex $start$.
2. Build an adjacency list for the undirected graph. For every edge (u, v, w) , add (v, w) to u 's list and (u, w) to v 's list.
3. Create a min-priority queue that stores the node and the weight of the edge used to reach it.
4. Insert the starting node into the priority queue with weight 0.
5. Create a visited set to keep track of vertices that have already been included in the MST.
6. Remove the minimum-weight entry from the priority queue. If its node is already visited, ignore it and continue.
7. Otherwise, mark the node as visited and add its edge weight to $totalWeight$.
8. For every unvisited neighbor of the selected node, insert the neighbor and its edge weight into the priority queue.
9. Repeat the process until the priority queue is empty. For a connected graph, all vertices are then included in the MST.
10. Return $totalWeight$ as the minimum total weight required to connect all nodes.
11. The starting node contributes weight 0 because no edge is needed to add the first vertex.
12. The visited set prevents the same vertex from being added more than once and avoids cycles in the resulting tree.
13. If multiple edges connect the same pair of vertices, all of them are inserted into the priority queue and the smallest useful edge is selected.

14. The adjacency list stores the graph efficiently and lets the algorithm examine all edges incident to the current vertex.
15. The priority queue always exposes the smallest available edge connecting the current tree to an unvisited vertex.
16. Continue selecting edges until n vertices have been visited. A connected MST therefore contains exactly $n - 1$ selected edges.
17. For the sample, start at node 1. The first available edges are 1–2 with weight 3 and 1–3 with weight 4; choose 1–2.
18. After visiting node 2, edge 2–5 with weight 2 becomes the smallest available useful edge; choose it.
19. Next choose 1–3 with weight 4, followed by 2–4 with weight 6.
20. The selected edges are 1–2, 2–5, 1–3, and 2–4, giving a total weight of $3 + 2 + 4 + 6 = 15$.
21. Time Complexity: $O((V + E) \log V)$ using an adjacency list and a min-priority queue.
22. Space Complexity: $O(V + E)$ for the adjacency list, visited set, and priority queue.

F. Code developed:

```
import java.io.*;
import java.util.*;

class Result {

    public static int prims(int n, List<List<Integer>> edges, int start) {
        // Build adjacency list
        Map<Integer, List<int[]>> graph = new HashMap<>();

        for (List<Integer> edge : edges) {
            int u = edge.get(0);
            int v = edge.get(1);
            int w = edge.get(2);

            graph.computeIfAbsent(u, k -> new ArrayList<>())
                .add(new int[]{v, w});

            graph.computeIfAbsent(v, k -> new ArrayList<>())
                .add(new int[]{u, w});
        }

        // Min-heap for edges (node, weight)
        PriorityQueue<int[]> pq =
            new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));

        Set<Integer> visited = new HashSet<>();
        pq.add(new int[]{start, 0});

        int totalWeight = 0;

        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int node = current[0];
            int weight = current[1];
```

```

        if (visited.contains(node)) continue;

        visited.add(node);
        totalWeight += weight;

        for (int[] neighbor : graph.getOrDefault(
            node, new ArrayList<>())) {
            if (!visited.contains(neighbor[0])) {
                pq.add(new int[]{neighbor[0], neighbor[1]});
            }
        }
    }

    return totalWeight;
}

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader =
            new BufferedReader(new InputStreamReader(System.in));

        BufferedWriter bufferedWriter =
            new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        String[] firstMultipleInput =
            bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int n = Integer.parseInt(firstMultipleInput[0]);
        int m = Integer.parseInt(firstMultipleInput[1]);

        List<List<Integer>> edges = new ArrayList<>();

        for (int i = 0; i < m; i++) {
            String[] edgeInput =
                bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

            List<Integer> edge = new ArrayList<>();
            for (String s : edgeInput) {
                edge.add(Integer.parseInt(s));
            }
            edges.add(edge);
        }

        int start = Integer.parseInt(bufferedReader.readLine().trim());
        int result = Result.prims(n, edges, start);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

G. HackerRank test case / Output screenshot (Mandatory):

The sample input and output for the HackerRank Prim's (MST) : Special Subtree problem are shown below.

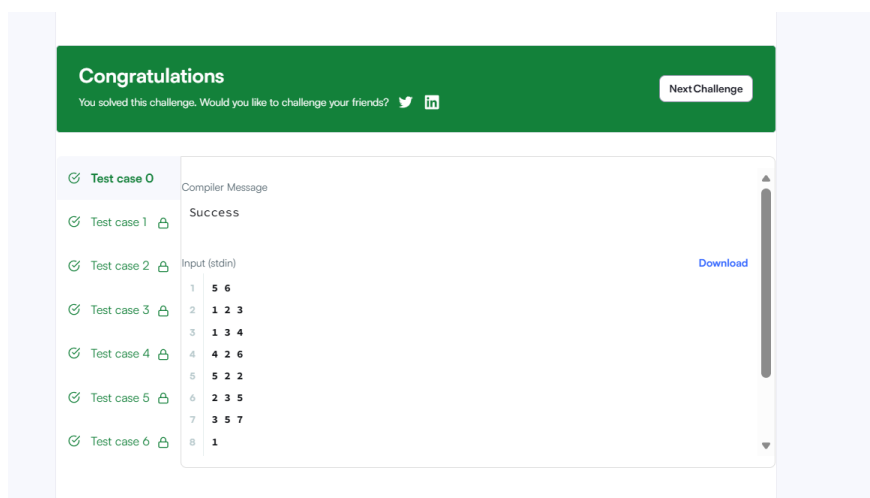


Figure: HackerRank sample input and expected output for Prim's MST.

H. Observation:

The assignment demonstrates the construction of a Minimum Spanning Tree using Prim's Algorithm. The graph is represented using an adjacency list so that all edges connected to a selected vertex can be examined efficiently.

The priority queue is the key data structure used in the implementation. It keeps the available edges ordered by weight, allowing the algorithm to select the smallest available connecting edge at each step. The starting vertex is inserted with weight 0, so it is included first without adding any edge cost.

The visited set prevents a vertex from being added more than once. When the same vertex can be reached through multiple edges, only the edge that reaches it first from the minimum-priority queue contributes to the MST. This prevents cycles and produces a spanning tree with minimum total weight.

For the sample input, the selected edges are 1–2 with weight 3, 2–5 with weight 2, 1–3 with weight 4, and 2–4 with weight 6. Therefore, all five vertices are connected and the total MST weight is 15.

Sample Graph Representation:

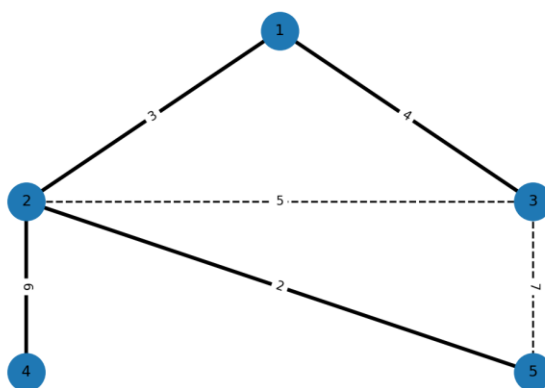


Figure: Sample weighted graph with the selected MST edges highlighted.